

Agentic Real-Time Processing

Day 7 designed the stream + provisioned it. Today Cascade authors the producer + consumer Lambdas. Sampled Bedrock enrichment (every Nth event, not all — costs!). STATE CONTEXT block in .windsurfrules v5 enters.

Day 7 → Day 8

Yesterday provisioned a 2-shard Kinesis stream, locked the 2-tier PK strategy, designed producer + consumer contracts. Today Cascade authors three Lambdas — producer + consumer (THIN: claim → StartExecution → ACK) + bedrock-enrich (Step Functions Task that does sampled Bedrock enrichment off the consumer SLA).

YESTERDAY

accelya-bookings-stream-<learner> (2 shards) ACTIVE

↓ consumed by ↓

TODAY

Producer Lambda PutRecords; THIN consumer ESM-triggered (claim → StartExecution → ACK; NO inline Bedrock call).

YESTERDAY

Two-tier PK: bulk + strict per-PNR

↓ used by ↓

TODAY

Producer chooses path per use-case. Cancellation events use bulk (PNR-bucketed).

YESTERDAY

stream_seen_keys_<learner> (TTL 7d)

↓ written by ↓

TODAY

Consumer claims pk=stream#stream#shard#sequence BEFORE any side-effect (Day 7 critical rule).

YESTERDAY

Lambda count: 6 of 10

↓ +3 today ↓

TODAY

+ producer + consumer + bedrock-enrich = 9 of 10. 1 left for D9 inference wrapper. (D11 PreTraffic Hook NOT a Lambda — alarm-based rollback only.)

YESTERDAY

STREAM CONTEXT block (.windsurfrules → v4)

↓ joined by ↓

TODAY

STATE CONTEXT block (.windsurfrules → v5) (which shard, what offset, what to retry, sampling cadence).

YESTERDAY

ESM config explicit (no AWS defaults)

↓ applied to ↓

TODAY

Lab 1 ESM created with RBIF, MaxRetry=3, MaxAge=3600, BisectBatch=true, ParallelizationFactor=1.

From contract to Cascade-authored Lambdas

THE CHALLENGE — TURN CONTRACT INTO CODE WITHOUT COUPLING THE CONSUMER

Day 7's contracts are paper. Today they become three production-ready Lambdas. The hard part isn't the code — Cascade scaffolds each in 5 minutes. The hard part is the architectural discipline:

- (a) Consumer must STAY THIN — claim → StartExecution → ACK. NO inline Bedrock or S3 writes from the consumer handler.
- (b) Bedrock enrichment lives in its OWN Lambda (bedrock-enrich) invoked from a Step Functions state machine — that decouples consumer SLA from Bedrock latency / throttle.
- (c) Sampling stays under 50-invocation cap; per-PNR ordering preserved end-to-end.

Why thin consumer? Day 9 adds your-model inference behind the SAME state machine via ASL Parallel. The consumer never has to know whether one or two enrichment branches run — it just StartExecutions and ACKs. The STATE CONTEXT block in .windsurfrules v5 keeps Cascade honest about this separation.

THE PAYOFF — END-TO-END STREAMING PIPELINE WITH DECOUPLED ENRICHMENT

By 03:30 today: producer Lambda PutRecords-ing booking events into the stream; THIN ESM-triggered consumer doing claim + StartExecution + ACK; Step Functions state machine `inference-pipeline-<learner>` runs the bedrock-enrich Task with sampled invocation; persists to enriched/ S3 zone. Day 9 adds the SageMaker wrapper as a SECOND Parallel branch in the SAME state machine — consumer code stays unchanged.

By the end of Day 8 you can...

BUILD

Cascade-author the producer Lambda using prompts/v11_stream_state.txt (11-layer prompt). Bulk + strict-order paths both implemented.

WIRE

Create the ESM with explicit lab-safe config (FunctionResponseTypes=[RBIF], ParallelizationFactor=1, MaxRetry=3, MaxAge=3600, BisectBatch=true). Provision the inference-pipeline state machine with one Task (bedrock-enrich).

BUILD

Cascade-author the THIN consumer Lambda — ESM-triggered, idempotency-claim, deterministic StartExecution on the inference-pipeline state machine, ACK. NO inline Bedrock or S3 calls from the consumer.

SAMPLE

Implement 1-in-N Bedrock enrichment in bedrock-enrich Lambda (default N=10 for lab; tune by event volume). Track invocations against 50-cap.

BUILD

Cascade-author the bedrock-enrich Lambda — Step Functions Task that does sampled (1-in-N) Bedrock InvokeModel + writes enriched/ S3. Decoupled from consumer SLA.

LAYER

Author STATE CONTEXT block in .windsurfrules v5 — shard, sequence_number, partial-batch retry rules, sampling cadence, deterministic SFN execution naming.

Four parts. Concepts → Design → Labs → Wrap.

01

Concept briefing

Six concepts. ESM mechanics. RBIF + sequence-number checkpoint (Day 7 rule applied). Sampled enrichment economics. Backpressure + IteratorAge. STATE CONTEXT for streams. Custom-exception ladder reused.

TIME

~30 min

02

Guided design

Three guided slides. THIN consumer + state-machine handoff architecture (ESM → consumer claim → StartExecution → ACK; SFN runs bedrock-enrich Task). Sampling cadence math (lives in bedrock-enrich Lambda). Cost-aware Bedrock invocation pattern.

TIME

~35 min

03

Three labs

Lab 1 — Cascade-author producer + THIN consumer + bedrock-enrich Lambda + inference-pipeline state machine (45 min). Lab 2 — wire ESM + observe partial-batch retry through SFN (45 min). Lab 3 — cost A/B: enrich-every vs enrich-sampled in the bedrock-enrich Lambda (45 min).

TIME

~135 min

04

Review & wrap

Five Day-8 mistakes. Cleanup respects ESM state (Day 9 reuses). Day-8 in numbers. Bridge to Day 9 — SageMaker Serverless inference; MODEL CONTEXT block in .windsurfrules v6.

TIME

~25 min

01

PART 1 · ~30 MINUTES

Concept briefing

Six concepts that make the consumer agentic, not just functional. ESM mechanics. RBIF + sequence-number checkpoint. Sampled enrichment economics (50 Bedrock invocations is tight). Backpressure. STATE CONTEXT for streams. Exception ladder reuse.

Per-shard polling + handler dispatch. Defaults are hostile; lab config overrides them.

Lambda's Kinesis ESM is doing more than 'trigger on records'. It polls per-shard, batches up to BatchSize records, invokes the handler, parses the response, and checkpoints. Each layer has a default that breaks at scale.

ESM lifecycle (per-shard)

```

1. Poll shard at GetRecords endpoint (default 5 polls/sec/shard)
2. Buffer records up to BatchSize OR MaxBatchingWindowInSeconds – whichever first
3. Invoke Lambda handler with {'Records': [...]}
4. Parse response:
  - {} or no batchItemFailures → ALL records succeeded → checkpoint advances past last
  - {'batchItemFailures': [{itemIdentifier:<seq>}, ...]}
    → checkpoint moves to LOWEST failed seq - 1
    → next poll re-reads from there (re-delivering records AFTER the failure too)
  - Lambda raised an unhandled exception → entire batch counts as failed
    → BisectBatchOnFunctionError=true halves + retries to isolate poison
5. Persist last-processed seq in Lambda's internal checkpoint store

DEFAULT ESM CONFIG (HOSTILE – DO NOT ACCEPT):
MaximumRetryAttempts:      -1  # retry until expiry
MaximumRecordAgeInSeconds: -1  # until expiry (24h default retention)
BisectBatchOnFunctionError: false
ParallelizationFactor:     1   # OK as default
StartingPosition:          LATEST # skips backlog!

LAB CONFIG (Day 7 lock):
MaximumRetryAttempts:      3
MaximumRecordAgeInSeconds: 3600
BisectBatchOnFunctionError: true
ParallelizationFactor:     1
StartingPosition:          LATEST (lab – avoids replaying D7's 10K test events into Bedrock
cap) # process from start
FunctionResponseType:      [ReportBatchItemFailures]
```

01

Per-shard poll

5 GetRecords/sec/shard default. ESM never goes idle.

02

Lowest-failed checkpoint

Day 7 critical rule: ESM rewinds to lowest, re-delivers AFTER too.

03

BisectBatchOnFunctionError

On unhandled exception: halve + retry. Isolates poison record.

04

TRIM_HORIZON for first read

LATEST skips backlog. Lab uses TRIM_HORIZON — replay-friendly.

05

Retry/Age finite

Default -1 means until expiry; lab uses 3 retries + 3600s age.

Day 7 rule applied: itemIdentifier = sequence_number; idempotency-claim BEFORE any side-effect.

Without idempotency, Kinesis RBIF semantics re-deliver records after a failure — the consumer double-processes. With idempotency, double-processing is a silent skip.

Consumer handler shape (sketch)

```
def handler(event, context):
    failures = []
    for record in event['Records']:
        seq = record['kinesis']['sequenceNumber']
        shard_id = record['eventID'].split(':')[0]
        try:
            # 1. STATUS-MODEL IDEMPOTENCY (avoids the 'mark-seen-then-crash' data loss)
            pk = f'stream#{STREAM}#{shard_id}#{seq}'
            status = idempotency.claim_in_progress(pk, lease_ttl_s=300)
            if status == 'COMPLETED':
                continue # silent skip - already processed
            if status == 'IN_PROGRESS_STALE':
                pass # crashed mid-handoff; re-StartExecution (recovery)
            if status == 'IN_PROGRESS_FRESH':
                failures.append({'itemIdentifier': seq})
                continue # another invocation owns it; fail fast/retry
            # else: status == 'CLAIMED' → we own it; proceed

            # 2. Decode + envelope-shape validate (lightweight; full schema lives downstream)
            envelope = json.loads(base64.b64decode(record['kinesis']['data']))
            schema.validate_envelope(envelope)

            # 3. ASYNC HANDOFF to Step Functions - DETERMINISTIC execution name
            # Consumer does NOT call Bedrock or write S3 inline. The state machine
            # runs the bedrock-enrich Task (sampled invocation lives there).
            exec_name = f'enrich-{{envelope["event_id"]}}'
            try:
                sfn.start_execution(
                    stateMachineArn=SM_ARN,
                    name=exec_name,
                    input=json.dumps({
                        'stream_pk': pk,
                        'correlation_id': envelope.get('correlation_id')
                    })
                )
            except Exception as e:
                failures.append({'itemIdentifier': seq, 'error': str(e)})
```

01

Status-model claim

CLAIMED → proceed. IN_PROGRESS_FRESH → fail-fast/retry.
IN_PROGRESS_STALE → reclaim. COMPLETED → skip. Mark
COMPLETED only AFTER StartExecution succeeds.

02

THIN consumer — NO inline Bedrock or S3

Consumer claims, StartExecutions, ACKs. Bedrock + persist live in
the state machine (bedrock-enrich Lambda Task). Decoupled from
consumer SLA.

03

Deterministic SFN exec name

f'enrich-{{event_id}}'; ExecutionAlreadyExists is idempotent for same
name+input WHILE running.

04

OLDEST-failure-only

Day 7 rule: reporting stale failures wastes work since checkpoint
rewinds.

05

stream COMPLETED ≠ enrichment COMPLETED

ACK only means 'handoff accepted'. State machine outputs + DLQ
track enrichment state independently.

50 Bedrock invocations is tight. Sample 1-in-N. Track usage. Sampling lives in the bedrock-enrich Step Functions Task, NOT the consumer.

If bedrock-enrich called Bedrock for every event, a 100-event burst burns 100 invocations — over the 50 cap. Sample 1-in-10 for lab; tune by event volume in production. Sampling decision happens inside the bedrock-enrich Lambda (it returns enrichment=null on skip), NOT in the THIN consumer.

Sampling math + cost-aware design — lives in bedrock-enrich Lambda

```
# Lab budget reality (per learner per session)
Bedrock cap:      50 invocations + 20K tokens
Used through D7:  ~10 invocations (Cascade-led design + Lab 3)
Day 8 budget:     ~40 invocations remaining
Day 8 events:     Lab 3 sends 100 cancellation events
                  Lab 1 demo: 10-20 records
                  Lab 2 partial-batch test: 10 records
                  ~120-130 events total in Day 8 labs

# Naive (enrich every): 130 invocations → BUSTS the cap
# Sampled 1-in-10:      13 invocations → fits with 27 left for D9

# Sampling rule lives in bedrock-enrich Lambda handler:
def handler(event, context):
    # Step Functions Task input = envelope
    if not should_enrich(event, sample_rate=int(os.environ.get('SAMPLE_RATE', '10'))):
        return {**event, 'enrichment': None, 'sampled': False}
    enrichment = bedrock_invoke(event) # may raise BedrockThrottleError
    return {**event, 'enrichment': enrichment, 'sampled': True}

def should_enrich(envelope, sample_rate=10):
    # Deterministic hash so same event always sampled the same way (replay-safe)
    import hashlib
    h = int(hashlib.md5(envelope['event_id'].encode()).hexdigest(), 16)
    return (h % sample_rate) == 0

# Production tuning:
Volume <100/min:    sample_rate=1 (enrich all)
Volume 100-1000/min: sample_rate=10
Volume >1000/min:   sample_rate=100 + tag enriched-event flag for downstream
```

01

1-in-N sampling

Deterministic hash on event_id. Replay-safe. Adjustable per volume.

02

Cap reality

50 invocations / session. Day 8 ≤35 left. Sample to fit.

03

Track all 3 metrics

Invocations + InputTokenCount + OutputTokenCount (Day 3 footnote).

04

Enrichment is NULLABLE

Most records have enrichment=NULL. Downstream queries handle it.

05

Prod tunes by volume

Lab default N=10; production scales N with throughput.

IteratorAge is the leading indicator of consumer lag. Alarm at 60s.

When the consumer can't keep up, IteratorAge metric grows. By the time downstream queries notice stale data, the lag may be hours. Alarm on IteratorAge > 60s; Day 10 wires the production version.

IteratorAge interpretation

```
IteratorAge (per shard, ms):
= (now - record_arrival_time at shard tail)
Healthy:      <1000 ms (1s)
Warming:      1000-10000 ms (1-10s)
Lag warning:  10000-60000 ms (10s-1min) → alert ops
Critical:     >60000 ms (>1min)      → page on-call
```

WHY LAG HAPPENS:

- Consumer Lambda timeout / OOM / cold-start storm
- Bedrock throttle (sampled enrichment helps)
- Downstream write-store throttle (DynamoDB / S3)
- Shard fanout exceeded (>5 enhanced consumers per shard)

MITIGATION (in order of cost):

1. Increase BatchSize (cheap; already at 100)
2. Increase ParallelizationFactor (>1 sacrifices ordering)
3. Add shards (resharding; control-plane operation)
4. Switch to enhanced fan-out (HTTP/2 push from Kinesis)

DAY 10 ALARM (preview):

```
aws cloudwatch put-metric-alarm \
  --alarm-name accelya-consumer-iterator-age \
  --metric-name IteratorAgeMilliseconds \
  --dimensions Name=StreamName,Value=accelya-bookings-stream-<learner> \
    Name=ShardId,Value=shardId-... \
  --statistic Maximum --period 60 --evaluation-periods 1 \
  --threshold 60000 --comparison-operator GreaterThanThreshold ...
```

01

IteratorAge in ms

GetRecords arrival - record arrival. Per-shard.

02

60s alarm

Day 10 wires production. Today the design names the metric.

03

Lag root causes

Lambda timeout, Bedrock throttle, downstream slow, fanout exceeded.

04

Mitigations cheapest first

BatchSize → ParallelizationFactor → reshard → enhanced fan-out.

05

Sampled enrich helps

Removes Bedrock from the per-record critical path.

Added to .windsurfrules v4 (becomes v5).

STREAM CONTEXT (Day 7) tells Cascade about the stream itself. STATE CONTEXT (Day 8) tells Cascade about per-event consumer state — which shard, what offset, what to retry, sampling cadence.

STATE CONTEXT layer template

```
## STATE CONTEXT (consumer state per record)
Triggered by:   ESM on accelya-bookings-stream-<learner>
Idempotency model — TWO GUARDS + STATUS LIFECYCLE:
  Guard A (Kinesis re-delivery): pk = f'stream#{STREAM}#{shard_id}#{sequence_number}'
  Guard B (producer-side dup):   pk = f'business#{event_type}#{booking_id}#{event_version}'
Status lifecycle (per pk):
  CLAIMED      → fresh; we own it; proceed with side-effects
  IN_PROGRESS  → another invocation owns it; lease_ttl_s=300
                  fresh: fail-fast (batchItemFailures, retry)
                  stale (>300s): reclaim; treat as recovery
  COMPLETED   → side-effects finished; silent skip on replay
CRITICAL: only mark COMPLETED AFTER all side-effects (persist) succeed.
Marking COMPLETED before persist creates the 'mark-seen-then-crash' data loss bug.
Idempotency table: stream_seen_keys-<learner> (TTL 7d)

Per-record state available to handler:
  shard_id      from record['eventID'].split(':')[0]
  sequence_number from record['kinesis']['sequenceNumber']
  partition_key  from record['kinesis']['partitionKey']
  arrival_time   from record['kinesis']['approximateArrivalTimestamp']
  data          base64-decoded JSON envelope

Partial-batch retry rules (Day 7 critical rules, restated):
- Idempotency-claim BEFORE any side-effect
- quarantine on ValidationError (NOT batchItemFailures)
- On unhandled exception: append seq to failures
- Return only OLDEST failure (checkpoint rewinds anyway)

Sampling cadence + HARD COST GUARD:
```

01

Layer slot

Between STREAM CONTEXT and CATALOG CONTEXT.

02

Per-record state

shard_id, sequence_number, PK, arrival_time, data — Cascade hardcodes against these.

03

Partial-batch rules

Day 7 rules restated: claim first, quarantine ≠ failure, oldest-only.

04

Sampling cadence

Deterministic hash; default 1-in-10; track 3 Bedrock metrics.

05

Failure routing

PipelineError ladder reused. BedrockThrottleError = transient + reduce sampling.

Day 4's PipelineError ladder applies to streams with one addition: BedrockThrottleError.

Streams reuse the existing exception design. Adding BedrockThrottleError keeps the ASL Catch routing pattern; downstream consumers don't need a new vocabulary.

Exception ladder (Day 4 + Day 8 addition)

```
## src/common/exceptions.py – Day 8 addition
class BedrockThrottleError(PipelineError):
    """Bedrock InvokeModel throttled (429); transient.
    Consumer adds to batchItemFailures; ESM retries.
    Trainer note: also reduce sample_rate dynamically if sustained."""

## Mapping to consumer behaviour
ValidationError      → quarantine/<learner>/stream_invalid/<reason>/
ManifestMismatch     → N/A on streams (no manifests)
MalformedRowError    → quarantine/<learner>/stream_invalid/parse_error/
BedrockThrottleError → batchItemFailures + EMF metric bedrock_throttled
TransientError       → batchItemFailures (generic catch-all)
ConfigError          → SNS escalate (Day 4 pattern)
```

01

1 new exception

BedrockThrottleError. Inherits PipelineError. Caught in consumer.

02

Same routing

Day 4 ladder unchanged; new exception slots in.

03

EMF on throttle

bedrock_throttled count → Day 10 dashboard.

04

Dynamic sample_rate

Sustained throttles → consumer lowers rate (Day 10 hardens).

05

Quarantine paths in S3

Day 4 paths reused:
quarantine/<learner>/stream_invalid/<reason>/.

Streaming-consumer vocabulary set

ESM mechanics. Day-7 RBIF + sequence checkpoint applied. Sampled enrichment economics. Backpressure indicators. STATE CONTEXT for streams. Exception-ladder reuse + 1 addition.

VOCABULARY

ESM, BatchSize, ParallelizationFactor, BisectBatchOnFunctionError, IteratorAge, SequenceNumberForOrdering, sampled enrichment.

WHAT'S NEW

STATE CONTEXT block (.windsurfrules → v5). Sampled-enrichment pattern (deterministic hash). BedrockThrottleError exception.

WHAT'S NEXT

Three guided design slides: consumer architecture, sampling cadence math, cost-aware Bedrock invocation pattern.

02

PART 2 · ~35 MINUTES

Guided design

Three guided design slides. Consumer architecture diagram. Sampling cadence math (1-in-10 lab default). Cost-aware Bedrock invocation pattern with EMF tracking.

End-to-end record path. Consumer is THIN by design (no inline Bedrock or S3); the inference-pipeline state machine owns enrichment + persist.

ARCHITECTURE FLOW

Consumer = 4 stages, all FAST. Bedrock + persist live downstream in the state machine.

```
[Kinesis stream]
  |
  | ESM polls (5 GetRecords/sec/shard)
  | batches up to 100 records
  |
  v
[THIN Consumer Lambda — accelya-consumer-<learner>]
  for record in event['Records']:
    1. Idempotency-claim (DDB stream_seen_keys)
    ← Day 7 rule
    pk = stream#<stream>#<shard>#<seq>
    duplicate? silent skip.
    2. base64 decode + envelope-shape validate
    fail? quarantine/<learner>/stream_invalid/.
    3. ASYNC HANDOFF — sfn.start_execution(
       stateMachineArn=SM_ARN,
       name=f'enrich-{event_id}', ← deterministic
       input={...envelope, stream_pk, correlation_id})
       ExecutionAlreadyExists? OK to skip (idempotent).
    4. Mark stream-PK COMPLETED (handoff accepted).

    return {'batchItemFailures': [oldest_failure_only]}
    ↓ async
[Step Functions: inference-pipeline-<learner>]
  StartAt: BedrockEnrich
  States:
    BedrockEnrich: Task → Lambda accelya-bedrock-enrich-
    <learner>
      (sampling 1-in-N lives HERE; Bedrock InvokeModel HERE)
      Retry: BedrockThrottleError → 2s × 3 backoff 2.0
      Catch: States.ALL → Quarantine state
      Next: Persist
    Persist: Task → Lambda accelya-persist-enriched-<learner>
      Writes enriched/<learner>/ JSON-Lines
      End: true
    Quarantine: Pass → SQS quarantine queue + S3 quarantine
    prefix
```

PER-RECORD METRICS (EMF) — split by Lambda

Each Lambda emits its own EMF block. Day 10 dashboard subscribes.

```
# Consumer (THIN — handoff metrics only):
records_in          count of records this
invocation
records_quarantined count routed to
quarantine (envelope-shape fail)
handoff_succeeded  count of successful
StartExecution
handoff_already_exists count of
ExecutionAlreadyExists (idempotent skip)
records_failed      count in
batchItemFailures
iterator_age_ms      max IteratorAge across
records (proxy lag)

# bedrock-enrich Lambda (in the state machine):
bedrock_invoked      count where Bedrock
InvokeModel actually called
bedrock_skipped      count where sampling
skipped (sampled=False)
bedrock_throttled    count of
BedrockThrottleError raised
input_tokens, output_tokens from Bedrock
response

# Tag dimensions:
StreamName + ShardId on consumer; ModelId on
bedrock-enrich.
```

WRITE PATTERN — enriched/<learner>/ (written by persist Lambda, not consumer)

JSON-Lines, partitioned by ingest_date + source_system. D5 catalog reads it.

```
Path: s3://accelya-airline-data-ap-south-
1/enriched/<learner>/
      ingest_date=YYYY-MM-
DD/source_system=GDS/
      events-<event_id>.jsonl.gz

Each line is a JSON object:
{ ...envelope (Day 7 schema)...,
  enrichment: { /* Bedrock output, or null */
},
  sampled: true|false,
// explicit flag
  enriched_at: 'ISO 8601' | null,
  consumer_seq: '<sequenceNumber>',
  consumer_shard: '<shard_id>',
  sfn_execution_arn: '...:execution:enrich-
<event_id>' // traceability
}
```

Day 9 ASL Parallel adds a SECOND branch (your-model SageMaker wrapper) feeding the SAME persist step. Consumer code unchanged — that's the point of THIN.

1-in-N deterministic sampling. Default N=10 for lab; tunable per volume.

DETERMINISTIC HASH (replay-safe)

MD5 on event_id; mod sample_rate. Same event → same decision on replay.

```
import hashlib
def should_enrich(envelope, sample_rate=10):
    h =
    int(hashlib.md5(envelope['event_id'].encode()).
        hexdigest(), 16)
    return (h % sample_rate) == 0

# WHY MD5 not random:
    random.random() rerolls per invocation. ESM-
    rewind = different sample.
    Idempotency-claim prevents double-WRITE;
    sampling decision must also be stable.
# Why event_id not sequence_number:
    sequence_number changes if upstream replays
    the SAME event with new SequenceNumber.
    event_id is the stable business identifier
    (envelope.event_id, Day 7 schema).
```

BUDGET MATH (lab session)

Day 8 needs to fit ~30 Bedrock invocations within remaining cap.

```
Cap (per session):      50 invocations + 20K
tokens
Used through D7:        ~15 invocations
(Cascade-led design + Lab 3 PutRecord smoke)
Day 8 reservation:      ~30 invocations
(sampled consumer)
Remaining for D9:        ~5 invocations
(SageMaker-side; minimal Bedrock)

Day 8 lab events:       ~120-130 records sent
across 3 labs
  Lab 1 demo:            10-20 records
  Lab 2 partial-batch:    10 records (deliberate
  poison)
  Lab 3 cost A/B:         100 records (split: 50
  enrich-every + 50 enrich-sampled)

If sample_rate=10:
  130 records × (1/10) = 13 enriched calls
  Plus Lab 3 enrich-every comparison: 50 calls
  Total: 63 calls — OVER cap.

Fix: Lab 3 uses 25 events (not 50) for enrich-
every:
  130 records × (1/10) = 13 + 25 = 38 calls.
  UNDER cap. Documented in lab brief.
```

PRODUCTION TUNING

Sample rate scales with volume. Adjust dynamically based on Bedrock throttle rate.

```
# Static tuning (from .windsurfrules per
environment):
  Volume <100/min → sample_rate=1 (enrich
  every; cheap)
  Volume 100-1k/min → sample_rate=10 (lab
  default)
  Volume >1k/min → sample_rate=100 + flag
  enriched events for downstream

# Dynamic tuning (Day 10 wires this; today the
design names it):
  if rolling_5min_throttle_rate > 0.05:
    sample_rate *= 2          # halve
    enrichment volume
  elif rolling_5min_throttle_rate == 0 and
  sample_rate > base_rate:
    sample_rate = max(base_rate, sample_rate
  // 2) # gradually relax
  # Persisted in DynamoDB
  consumer_state_<learner> table (Day 10
  introduces)
```


Wrap every InvokeModel call. Track all 3 metrics. Exception → known mapping.

WRAPPER PATTERN

Cascade-authored. Tracks tokens; raises domain exceptions on AWS errors.

```
import boto3
from botocore.exceptions import ClientError

bedrock = boto3.client('bedrock-runtime',
    region_name='ap-south-1')
MODEL_PROFILE = 'apac.anthropic.claude-3-5-sonnet-20241022-v2:0'

def bedrock_enrich(envelope: dict) -> dict:
    """Calls Bedrock; raises
    BedrockThrottleError on throttle.
    Returns parsed enrichment dict."""
    prompt = build_prompt(envelope) # caller-
    defined; uses MODEL CONTEXT layer
    try:
        response = bedrock.invoke_model(
            modelId=MODEL_PROFILE,
            body=json.dumps({
                'anthropic_version': 'bedrock-
2023-05-31',
                'max_tokens': 256,
                'messages': [{'role': 'user',
'content': prompt}]
            }),
            contentType='application/json',
        )
    except ClientError as e:
        code = e.response['Error']['Code']
        if code in ('ThrottlingException',
'ServiceUnavailable'):
            # ServiceUnavailable:
            extra={'code': code})
```

TOKEN TRACKING

Bedrock returns usage in the response. Emit per-call EMF for Day 10.

```
# After invoke_model:
usage = body.get('usage', {})
emf({
    'bedrock_input_tokens':
usage.get('input_tokens', 0),
    'bedrock_output_tokens':
usage.get('output_tokens', 0),
    'bedrock_invocation_ms': elapsed_ms,
}, dimensions={
    'ModelId': MODEL_PROFILE,
    'Function': context.function_name,
})

# Day 10 dashboard sums these against 50-inv /
20K-token cap.
# Consumer also persists running totals in
DynamoDB consumer_state_learner>
# (Day 10 introduces) for cross-invocation
visibility.
```

PROMPT BREVITY (cost discipline)

Each Bedrock call uses ~200 tokens in for ~100 tokens out. Tight prompts matter.

```
# Bad enrichment prompt (~600 tokens in):
"""You are an airline cancellation analyst.
Given the following cancellation
event with PNR, fare class, OD pair,
ticket number, ... [verbose]
Please analyze and return a JSON with:
cancellation_reason_category,
refund_priority_tier,
suspected_fraud_risk, recommended_outreach.
Be detailed and provide explanations for
each field."""

# Good enrichment prompt (~150 tokens in):
"""Given event: {airline}{pnr}
fare={fare_class} {od_pair}
reason={reason_code}
Output JSON only:
{category:
<weather|operational|passenger|other>,
refund_priority: <high|med|low>,
fraud_risk: <0..1>}"""

# Both produce comparable enrichment quality.
# Lab 3 cost A/B should also include prompt
brevity comparison if time permits.
```

Consumer architecture ratified — THIN consumer + state-machine handoff

Consumer is THIN by design (claim → StartExecution → ACK). State machine runs the bedrock-enrich Task; sampling lives there. EMF metrics split per Lambda.

THIN CONSUMER, FAT STATE MACHINE

Consumer = 4 fast stages. Bedrock + persist = SFN tasks.
Decoupled SLA.

SAMPLE 1-IN-10 IN bedrock-enrich

Deterministic hash on event_id; sampling lives in the
bedrock-enrich Lambda (NOT the consumer).

DAY 9 SUPERSET

D9 adds your-model SageMaker wrapper as a SECOND ASL
Parallel branch. Consumer code unchanged.

03

PART 3 · ~135 MINUTES

Three labs

Lab 1 (45 min) — Cascade-author producer + THIN consumer + bedrock-enrich Lambda + inference-pipeline state machine. Lab 2 (45 min) — wire ESM with explicit lab config + observe partial-batch retry through SFN. Lab 3 (45 min) — cost A/B: 25 events enrich-every vs 25 events enrich-sampled (toggle `SAMPLE_RATE` on bedrock-enrich Lambda).

Cascade-author producer + THIN consumer + bedrock-enrich + state machine

BRIEF

Pair-work. Use prompts/v11_stream_state.txt (the 11-layer with STATE CONTEXT). Cascade scaffolds FOUR artefacts: (1) src/producer/handler.py (bulk path; PutRecords with PNR-bucketed PK); (2) src/consumer/handler.py (THIN: claim → StartExecution → ACK; NO inline Bedrock or S3); (3) src/bedrock_enrich/handler.py (Step Functions Task; sampled 1-in-N Bedrock InvokeModel + persist call); (4) infra/inference-pipeline.asl.json (StartAt: BedrockEnrich → Persist; Catch → Quarantine). Plus src/common/exceptions.py (+ BedrockThrottleError). Lambda count cumulative: 6 → 9 (+ producer + consumer + bedrock-enrich). 1 left for D9 wrapper.

TIME

45 min

LEVEL

Working+

DELIVERABLE

src/producer/handler.py + src/consumer/handler.py
(THIN) + src/bedrock_enrich/handler.py
infra/inference-pipeline.asl.json
src/common/exceptions.py (+
BedrockThrottleError)
scripts/deploy_m8_lambdas.sh +
scripts/deploy_m8_state_machine.sh
/reviews/HITL_REVIEW-m8-streaming.md (6 dims +
per-PK ordering = 7th)

Six steps. Branch → v11 prompt → Cascade-author Lambdas → deploy → verify → MR.

1

Branch + 11-layer prompt template

```
export LEARNER=jai
git checkout main && git pull --ff-only
git checkout -b module-8-$LEARNER
```

EXPECTED

On module-8-<learner>. v11_stream_state.txt has 11 layers; STATE CONTEXT visible.

2

Cascade-author producer Lambda (PutRecords + partial-failure handling)

```
# In Windsurf chat: attach prompts/v11_stream_state.txt
# TASK: Scaffold src/producer/handler.py per STREAM CONTEXT bulk path.
#       Input: {events: [<envelope>, ...]}; Output: {submitted_count, failed_count, failed_records}.
#       PutRecords (<5000) with PartitionKey=f'<inline>.<data>.<bucket> hash(<nn>)%01'
```

EXPECTED

src/producer/handler.py implements explicit PutRecords partial-failure handling.

3

Cascade-author THIN consumer + bedrock-enrich Lambda + inference-pipeline ASL

```
# Same chat (continues context):
# TASK 1: Scaffold src/consumer/handler.py per STATE CONTEXT (THIN consumer):
#       4 stages: claim → envelope-validate → start_execution → ACK.
#       Deterministic event name f'<enrich>-<event id>'
```

EXPECTED

src/consumer/handler.py is THIN (claim → StartExecution → ACK).
src/bedrock_enrich/handler.py has sampling + Bedrock invoke.
infra/inference-pipeline.asl.json valid. src/common/exceptions.py has BedrockThrottleError.

4

Deploy 3 Lambdas + state machine

```
# Deploy producer + THIN consumer + bedrock-enrich:
for L in producer consumer bedrock_enrich; do
  (cd src/$L && zip -r /_/$L.zip && cd -)
#   # Task should_enrich() via deterministic msg hash on event_id.
```

EXPECTED

3 Lambdas Active (cumulative 9 of 10). State machine ARN exported as \$SM_ARN. 1 left for D9 wrapper.

5

Pair-HITL on the consumer (7 dims)

```
# 7 dims: Correctness, Safety, Idempotency, Observability, Cost,
#       Per-PK Ordering (Day 7 added), Schema Discipline (Day 5 added).
# Most watched: Idempotency FIRST? Quarantine # batchItemFailures? OLDEST only?
# TASK 4: Add BedrockThrottleError to src/common/exceptions.py
```

EXPECTED

/reviews/HITL_REVIEW-m8-streaming.md scored ≥17/21 across 7 dims.

6

Commit + open MR

```
git add src/ scripts/ prompts/v11_stream_state.txt reviews/
git commit -m "module-8: producer + consumer Lambdas (Lab 1)"
git push origin module-8-$LEARNER
```

EXPECTED

MR open with 4 files (producer, consumer, exceptions, deploy script).

Success criteria & common gotchas

✓ YOU ARE READY IF...

- Producer + THIN consumer + bedrock-enrich Lambda + state machine + BedrockThrottleError committed; Lambda count 9 of 10
- PartitionKey formula matches PNR-bucketed bulk path
- Consumer is THIN: claim → envelope-validate → start_execution (deterministic name) → ACK; NO inline Bedrock or S3
- bedrock-enrich Lambda owns sampling (SAMPLE_RATE env) + Bedrock InvokeModel + persist call
- EMF metrics split per Lambda: handoff_* on consumer; bedrock_* + sampled on bedrock-enrich
- HITL review scored ≥17/21 across 7 dims
- MR open

⚠ TOP GOTCHAS

Inline Bedrock call in consumer

Day 9 explicitly forbids this. Couples consumer SLA to Bedrock latency. Move it to bedrock-enrich Lambda from the start.

Side-effect before idempotency claim

Re-delivery on lowest-checkpoint rewinds the consumer. Without claim-first, double-process.

Quarantine in batchItemFailures

Day 4 trap from streams. Envelope-shape fails are silent; not batch failures.

Non-deterministic SFN exec name

Without f'enrich-{event_id}', a re-delivered record creates a SECOND state-machine run for the same event. Use the deterministic name + treat ExecutionAlreadyExists as success.

Reporting all failures

Wastes work since checkpoint rewinds. Return only OLDEST.

Wire ESM + observe partial-batch retry through state machine

BRIEF

Solo. Create ESM with the Day-7 explicit config (RBIF, ParallelizationFactor=1, MaxRetry=3, MaxAge=3600, BisectBatch=true). Send 10 records: 8 valid + 2 deliberately poison-envelope. Trace via Logs Insights + Step Functions: confirm 8 successful StartExecutions (handoff_succeeded EMF=8); 2 quarantined (envelope-shape fail; NOT in batchItemFailures); 8 state-machine executions visible in SFN console; persist Lambda wrote 8 records to enriched/. Verify deterministic exec name handles re-delivery (re-trigger same event_id → ExecutionAlreadyExists → idempotent skip).

TIME

45 min

LEVEL

Working+

DELIVERABLE

ESM created with explicit config
8 state-machine executions visible (one per non-poison event) — confirms async handoff worked
[/reviews/m8-partial-batch-evidence.md](#) (Logs Insights + SFN list-executions output + idempotency table count)

Six steps. Create ESM → send 10 records → trace → quarantine check → idempotency table check → document.

1

Create ESM with explicit config

```
STREAM_ARN=$(aws kinesis describe-stream --stream-name accelya-bookings-stream-$LEARNER \
--region ap-south-1 --query 'StreamDescription.StreamARN' --output text)
```

EXPECTED

ESM Created. FunctionResponseTypes=[ReportBatchItemFailures]. State Enabled.

2

Send 10 records (8 valid + 2 poison) via producer

```
python3 -m scripts.gen_m8_test_batch --rows 8 --poison 2 --out /tmp/m8-test.json
aws lambda invoke --function-name accelya-producer-$LEARNER \
--payload file:///tmp/m8-test.json /tmp/m8-out.json --region ap-south-1
```

EXPECTED

submitted_count=10. Lambda invokes producer; producer PutRecords 10 records.

3

Trace via Logs Insights

```
# Logs Insights query for accelya-consumer-$LEARNER:
fields @timestamp, seq, event, reason
| filter @message like /event=record/ or
# Where DLO_ARN is an SQS queue stream-dlo-<learner> created BEFORE the ESM.
```

EXPECTED

8 records succeeded; 2 quarantined for parse_error / schema_drift; oldest_failure may show 0-1 (depends on throttle).

4

Verify quarantine + S3 enriched/

```
aws s3 ls s3://accelya-airline-data-ap-south-1/quarantine/$LEARNER/stream_invalid/ \
--recursive --region ap-south-1
aws s3 ls s3://accelya-airline-data-ap-south-1/enriched/$LEARNER/ \
# verify: this confirms an enriched record
```

EXPECTED

2 files in quarantine/<learner>/stream_invalid/. enriched/<learner>/ has 8 records.

5

Verify idempotency table claims

```
aws dynamodb scan --table-name stream_seen_keys_$LEARNER \
--region ap-south-1 \
--filter-expression "begins_with(pk, :p)" \
--expression-attribute-values '{":p":{"S":"pkname"}}'
```

EXPECTED

10 PKs (one per sequence number) with outcome 'processed' or 'quarantined'.

6

Document + commit

```
# /reviews/m8-partial-batch-evidence.md captures Logs Insights output + counts.
git add reviews/m8-partial-batch-evidence.md
git commit -m "module-8: ESM partial-batch evidence (Lab 2)"
git push
```

EXPECTED

MR shows 2 commits.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- ESM created with all explicit config flags (StartingPosition=LATEST)
- 8 valid records reach enriched/
- 2 poison records reach quarantine/<reason>/
- Lambda return shape verified: batchItemFailures = [{itemIdentifier: <oldest_failed_sequence_number>}]
- 10 stream-PKs in stream_seen_keys with status COMPLETED for the 8 successes
- Re-delivered records (after checkpoint rewind) are idempotency-skipped — verify by inducing one transient failure mid-batch and observing the rewind doesn't double-write to enriched/
- Evidence file captures Logs Insights output AND the verification of the rewind safety
- MR shows 2 commits

⚠ TOP GOTCHAS

Forgetting --bisect-batch-on-function-error

Default false. Without it, poison records re-deliver entire batch infinitely.

Wrong starting position

Default LATEST skips your test records that landed before ESM created.

ESM scope too broad

Don't accidentally point ESM at validated/ S3 prefix (different ESM type).

Forgetting --function-response-types

Day 4 trap; same here. Verify with list-event-source-mappings.

Producer Lambda timeout too short

PutRecords retry can take ~10s under throttle. 60s timeout is safe.

Cost A/B — enrich-every vs enrich-sampled (toggle SAMPLE_RATE on bedrock-enrich Lambda)

BRIEF

Pair-work. Toggle the bedrock-enrich Lambda's SAMPLE_RATE env var: send 25 events with SAMPLE_RATE=1 (every) then 25 events with SAMPLE_RATE=10 (sampled). Capture Bedrock invocation count + token totals per CloudWatch. Document the 10× difference. Confirm sampled run produced records with sampled=false on most + sampled=true on ~1-in-10 (deterministic, replay-safe). NOTE: SAMPLE_RATE lives on bedrock-enrich Lambda (the Step Functions Task), NOT on the THIN consumer.

TIME

45 min

LEVEL

Working

DELIVERABLE

/reviews/m8-enrichment-cost-ab.md (invocations + tokens for both runs + ratio + state-machine execution counts)

Six steps. Capture pre-test metrics → send 25 enrich-every → measure → send 25 enrich-sampled → measure → ratio.

1

Capture pre-test Bedrock metrics

```
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $(date -u -d 75 minutes ago) --end-time $(date -u -d 75 minutes ago)
```

EXPECTED

Baseline numbers captured for the day.

2

Run 25 events with sample_rate=1 (enrich every)

```
# Override consumer's sample_rate via Lambda env var:
aws lambda update-function-configuration \
  --function-name accelya-bedrock-enrich-$LEARNER \
  --environment Variables=(SAMPLE_RATE=1) --region ap-south-1
```

EXPECTED

25 Bedrock invocations triggered.

3

Capture metrics for run A

```
for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $(date -u -d 75 minutes ago) --end-time $(date -u -d 75 minutes ago)
```

EXPECTED

Δ Invocations ≈ 25.

4

Run 25 events with sample_rate=10

```
aws lambda update-function-configuration \
  --function-name accelya-bedrock-enrich-$LEARNER \
  --environment Variables=(SAMPLE_RATE=10) --region ap-south-1
```

EXPECTED

Only 2-3 Bedrock invocations (deterministic hash).

5

Capture metrics + compute ratio

```
for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $(date -u -d 75 minutes ago) --end-time $(date -u -d 75 minutes ago)
```

EXPECTED

Ratio ≈ 8-10× (1-in-10 sample).

6

Document + reset sample_rate + commit

```
# /reviews/m8-enrichment-cost-ab.md:
# Run A (sample_rate=1): invocations=25, in_tokens=<X>, out_tokens=<Y>
# Run B (sample_rate=10): invocations=2-3, in_tokens=<X/8>, out_tokens=<Y/8>
# Ratio: ~10× cost reduction. Conclusion: sample for production.
# Reset sample_rate (Day 9 wants the default):
aws lambda update-function-configuration \
  --function-name accelya-bedrock-enrich-$LEARNER \
  --environment Variables=(SAMPLE_RATE=1) --region ap-south-1
```

EXPECTED

MR shows 3 commits. SAMPLE_RATE reset to 10 for D9.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- Run A and Run B both completed with metric capture
- Δ invocations approximately match expected ratio (~10×)
- Sampled run shows enrichment=NULL on most records, populated on ~1/10 (deterministic)
- Cost A/B documented
- SAMPLE_RATE reset to 10
- MR shows 3 commits

⚠ TOP GOTCHAS

Bedrock CW metric lag

30-60s delay. Sleep before query.

SAMPLE_RATE on the wrong Lambda

SAMPLE_RATE lives on bedrock-enrich, NOT the THIN consumer. Updating consumer env var has no effect on enrichment.

random.random() snuck in

Cascade may have used random instead of MD5 hash in bedrock-enrich. Inspect should_enrich(); replays must be deterministic.

Forgot reset

Leaving SAMPLE_RATE=1 on bedrock-enrich burns Day 9's Bedrock budget.

Counting wrong dimension

ModelId emits as the inference profile string; verify in console first (Day 7's note).

Code shipped. Branches pushed. Time to step back.

04

PART 4 · ~25 MINUTES

Review & wrap

Five Day-8 streaming-consumer mistakes. Cleanup respects ESM (Day 9 reuses). Day-8 in numbers. Bridge to Day 9 — SageMaker Serverless inference. MODEL CONTEXT block in .windsurfrules v6.

Five Day-8 streaming-consumer mistakes — and what to do instead

01

Inline Bedrock call in the consumer (couples SLA)

MISTAKE Consumer calls `bedrock.invoke_model` inline. Bedrock cold-start / throttle now sits on the consumer's per-batch deadline; ESM rewinds + double-processes when Bedrock takes >30s.

DO INSTEAD

THIN consumer: `claim` → `StartExecution` → `ACK`. Bedrock lives in the `bedrock-enrich` Step Functions Task. D9 superset proves the pattern by adding a `SECOND` Parallel branch (your-model SageMaker) without touching the consumer.

02

Marking COMPLETED before StartExecution succeeds (data loss)

MISTAKE Consumer marks `pk=COMPLETED` as soon as it's claimed; then crashes before `StartExecution`. Replay sees `COMPLETED` → silent skip → event never enriched.

DO INSTEAD

Status model: `CLAIMED` → `start_execution` → mark `COMPLETED`. Stale `IN_PROGRESS` reclaimed for recovery.

03

Non-deterministic SFN execution name

MISTAKE Consumer uses `uuid4()` for `exec_name`. Re-delivered records spawn duplicate state-machine runs; double-enrich; cap blown.

DO INSTEAD

`exec_name = f'enrich-{{event_id}}'`. `ExecutionAlreadyExists` treated as idempotent skip. Same name+input is idempotent WHILE running per Step Functions Standard semantics.

04

`random.random()` sampling in bedrock-enrich

MISTAKE Replays re-roll the dice. Same event sometimes enriched, sometimes not.

DO INSTEAD

Deterministic hash on `event_id` (`MD5 % SAMPLE_RATE`). Replay-safe.

05

ESM defaults accepted

MISTAKE `MaximumRetryAttempts=-1 + MaximumRecordAgeInSeconds=-1` = retry until expiry. Lab cap blown.

DO INSTEAD

Always pass `--maximum-retry-attempts 3 --maximum-record-age-in-seconds 3600 --bisect-batch-on-function-error`.

End-of-module cleanup

CLEANUP CHECKLIST

- Branch pushed: module-8-<learner>; MR with 3 commits; pair partner tagged
- Producer + THIN consumer + bedrock-enrich Lambdas Active (count cumulative: 9 of 10)
- inference-pipeline-<learner> state machine ACTIVE with BedrockEnrich → Persist + Catch → Quarantine
- ESM Active with explicit config — DO NOT DELETE (Day 9 adds your-model branch INSIDE the state machine; consumer untouched)
- Stream + stream_seen_keys + IAM role preserved (D7 inheritance)
- enriched/<learner>/ S3 prefix created with first events (Day 9 reads as input)
- Bedrock invocations: ~25 of 50 used (sampled enrichment + Cascade scaffolding); 25 left for D9/D10/D11
- SAMPLE_RATE on bedrock-enrich reset to 10 (don't burn Day 9 budget)

```
# Verify nothing critical was deleted
aws lambda list-event-source-mappings \
  --function-name accelya-consumer-$LEARNER --region ap-south-1 \
  --query
'EventSourceMappings[*].{State:State,FRT:FunctionResponseType,MR:MaximumRetryAttempts}'

aws lambda list-functions --region ap-south-1 \
  --query "length(Functions[?contains(FunctionName,'$LEARNER')])"
# Expected: 9 (foundation 6 + producer + consumer + bedrock-enrich)

aws stepfunctions describe-state-machine --state-machine-arn $SM_ARN \
  --region ap-south-1 --query 'status'
# Expected: ACTIVE

# Bedrock budget (3 metrics):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $(date -u -d 'today 00:00' +%FT%T) \
    --end-time $(date -u +%FT%T) --period 3600 --statistics Sum \
    --region ap-south-1
done
```

Trainer note: Day 9 adds the SageMaker wrapper as a SECOND Parallel branch in the SAME inference-pipeline state machine. Consumer code unchanged. Lambda count 9 of 10 — 1 left (D9 wrapper). Bedrock 25/50 used; D9 needs ~5; D10/D11 unaffected.

What you actually shipped

9/10

LAMBDA

+3 today (producer + THIN consumer + bedrock-enrich). 1 left for D9 wrapper.

1

STATE MACHINE

inference-pipeline-<learner>.
BedrockEnrich → Persist; Catch → Quarantine. D9 forks BedrockEnrich into ASL Parallel.

10×

ENRICH COST CUT

Sampled 1-in-10 vs every (toggle on bedrock-enrich Lambda's SAMPLE_RATE env).

11

PROMPT LAYERS

+ STATE CONTEXT for streams. Includes deterministic SFN exec naming + THIN consumer rule.

Day 9 — Real-Time ML/DL Inference (SageMaker Serverless)

Today the consumer hands off to a state machine that runs bedrock-enrich. Tomorrow your OWN model joins via SageMaker Serverless — added as a SECOND ASL Parallel branch in the SAME state machine. Consumer code stays unchanged. MODEL CONTEXT block in .windsurfrules v6 enters; 8th HITL dim Confidence Handling.

WRAPPER LAMBDA

+1 Lambda (count → 10 of 10 — final cap)

src/predict/handler.py wraps `invoke_endpoint` with cold-start retry, throttle catch, confidence threshold. Becomes the second Parallel branch alongside bedrock-enrich.

12TH PROMPT LAYER

MODEL CONTEXT (which model, input shape, confidence threshold, fallback)

Cascade scaffolds the wrapper with sandbox-aware caps (1 endpoint, no GPUs).

SANDBOX BUDGET

1 SageMaker endpoint + ~5 Bedrock invocations

Cumulative: 10 of 10 Lambdas (final cap reached); 1 of 1 SageMaker endpoints; ~30 of 50 Bedrock used.

Day 9 branch: module-9-<learner>. Inherits .windsurfrules v5 (with STATE CONTEXT). Bring your consumer + bedrock-enrich + state machine.

Streaming pipeline live. Consumer is agentic.

PRODUCER + CONSUMER

Both Cascade-authored. PutRecords bulk path; ESM-triggered consumer with 5-stage flow.

SAMPLED ENRICHMENT

1-in-10 deterministic sampling. Ratio proven 10× in Lab 3 evidence. Cap-aware.

.WINDSURFRULES v5

STATE CONTEXT block added. BedrockThrottleError joins exception ladder.

Day 9 starts at 09:00. Bring your consumer + enriched/. SageMaker Serverless enters; MODEL CONTEXT block in .windsurfrules v6 joins the .windsurfrules v5 you used today.